

Project Rubrics

Cloud Computing Course 2026

Applications

1- Customer Support Ticketing System

Description

A company aims to develop a Customer Support Ticketing System to manage customer inquiries, issues, and support requests efficiently. The system allows users to submit tickets, track their status, and receive responses from support agents. Support staff can manage, prioritize, and resolve tickets while maintaining a history of interactions. To support scalability, maintainability, and continuous integration/deployment, the system is developed using modern DevOps tools such as Docker, Docker-Compose, and Kubernetes.

Architecture

- Simplified ticketing platform organized into multiple microservices (at least 3 services).
- Services shall be designed to be independent, loosely coupled, and responsible for specific business functionalities such as Support Service (Handles ticket assignment, responses, and resolution workflows), Ticket Service (Manages ticket creation, updates, and status tracking), Reporting Service (Generates reports on ticket performance and response times), and Notification Service (Sends updates and alerts regarding ticket status).
- These services are independently deployable and communicate with each other through APIs (using lightweight protocols such as HTTP).

2- Event Management System

Description

An organization aims to develop an Event Management System to plan, organize, and manage events such as conferences, workshops, and seminars. The system allows event organizers to create events, manage schedules, and handle registrations, while users can browse events and reserve tickets. It also supports payment processing and notification features for reminders and updates. To support scalability, maintainability, and continuous integration/deployment, the system is developed using modern DevOps tools such as Docker, Docker-Compose in different environments (testing, development and production), and Kubernetes.

Architecture

- Simplified event management platform organized into multiple microservices (at least 3 services).

- Services shall be designed to be independent, loosely coupled, and responsible for specific business functionalities such as User Management Service (Handles user registration, login, and profiles), Registration Service (Handles event bookings and participant management), Event Service (Manages event creation, schedules, and details), and Notification Service (Sends event reminders and updates).
- These services are independently deployable and communicate with each other through APIs (using lightweight protocols such as HTTP).

3- Food Delivery System

Description:

A food delivery company aims to develop a scalable Food Delivery System to connect customers, restaurants, and delivery personnel through a unified platform. The system allows users to browse restaurant menus, place orders, and track deliveries in real time. Restaurants can manage their menus and orders. The system needs to be highly scalable, secure, and reliable to handle sensitive patient data. To ensure scalability, reliability, and efficient deployment, you need to utilize Docker, Docker-Compose, Kubernetes in the development and deployment processes.

Architecture:

- Simplified food delivery platform divided into a collection of services. (at least 3 services)
- Services shall be designed to be independent, loosely coupled, and responsible for specific business functionalities such as User Management Service (Handles customer and delivery personnel registration, login, and profiles) , Restaurant Service (Manages restaurant data, menus, and item availability), Order Service (Handles order creation, updates, and order tracking), Payment Service (Processes payments and manages transaction records).
- These services are independently deployable and communicate with each other through APIs (using lightweight protocols such as HTTP).

4- Job Portal System (Mini LinkedIn Jobs)

Description:

An organization aims to build a Job Portal System to streamline the recruitment process by connecting employers and job seekers on a single platform. The system allows companies to post job opportunities, manage applications, and review candidates, while job seekers can create profiles, search for jobs, and apply directly through the platform. The system needs to be highly scalable, secure, and reliable to handle sensitive patient data. To ensure scalability, reliability, and efficient deployment, you need to utilize Docker, Docker-Compose, Kubernetes in the development and deployment processes.

Architecture:

- Simplified Job Portal System platform divided into a collection of services. (at least 3 services)
- Services shall be designed to be independent, loosely coupled, and responsible for specific business functionalities such as User Management Service (Handles registration, login, and profile management for job seekers and employers), Job Service (Manages job postings, job details, and company listings), Application Service (Handles job applications, status tracking, and candidate submissions), Notification Service (Sends alerts regarding job applications and updates).
- These services are independently deployable and communicate with each other through APIs (using lightweight protocols such as HTTP).

General Requirements:

1. **Project must be hosted on a Linux machine with any distribution of your choice (not Windows and not macOS).**
2. **Project must have multiple containers that communicate with each other.**
3. **Containerization:**
 - Each service shall be containerized using Docker to encapsulate its dependencies and ensure consistency across different environments. (e.g., frontend, backend, database, ...etc.)
 - Custom Docker images that build the app shall be optimized for size and performance to facilitate efficient deployment and scaling.
 - Application should use at least 3 different images (multiple instances of the same image is not counted)
4. **Multiple Environments Definition:**
 - The system should support **different configurations** for each environment: (Development, Testing, and Production).
 - Developers shall be able to spin up all configured environments at the same time on a single machine.
 - Docker-Compose shall be utilized to define and manage multi-container development, staging and production environments.
 - Developers shall be able to spin up the entire application stack locally using Docker-Compose for seamless integration testing and development.
5. **Orchestration with Kubernetes:**
 - Containerized services shall be deployed and managed using Kubernetes for efficient orchestration.
6. **Documentation:**
 - Comprehensive documentation shall be provided for explaining the project's structure, components, and how to set up development environments, and deploying the platform.

[Bonuses]

Functionality

- **Monitoring and Logging:**
 - Monitoring and logging shall be implemented using tools like Prometheus and Grafana, or ELK for monitoring containerized services and Kubernetes clusters.
 - Logs shall be centrally collected and stored for troubleshooting and analysis purposes.
- **Asynchronous Communication:**
 - Services communicate with each other asynchronously over a network using messaging systems like Kafka or RabbitMQ.